

Guía Ejercicios Practicos

Estructura de datos

1 de octubre de 2026

1. Listas Enlazadas

1.1. Ejercicio: Lista Estudiantes

En una asignatura de 2.^o año, se dispone de una lista enlazada con los 35 alumnos que integran el curso. La lista original se encuentra ordenada alfabéticamente por nombre. De cada alumno se conoce: Nombre (string) y Promedio general (float) obtenido al finalizar el año. Desarrollar un programa que recorra la lista original y distribuya/gestione a los estudiantes según su promedio, manteniendo el orden alfabético en todas las estructuras involucradas:

- **Lista de Julio:** Copiar/mover a una nueva lista ordenada a los alumnos que rinden examen en el turno de Julio (promedio en el rango $[4,0, 5,0)$, es decir: $\geq 4,0$ y $< 5,0$).
- **Lista de Agosto:** Copiar/mover a otra nueva lista ordenada a los alumnos que rinden examen en el turno de Agosto (promedio $< 4,0$).

Es importante considerar que los alumnos aprobados (promedio $\geq 5,0$) no rinden examen, por lo que deben ser eliminados de la lista original, liberando la memoria dinámica correspondiente. Al finalizar el procesamiento descrito el programa debe imprimir por pantalla:

- El listado completo de alumnos que rinden en Julio.
- El listado completo de alumnos que rinden en Agosto.

1.2. Ejercicio: Ticket supermercado

Un supermercado nos pide que hagamos una pequeña aplicación que almacene los productos pasados por el escaner. Para representar cada producto, se debe crear una clase Producto. Cada producto debe contener, como mínimo, los siguientes datos:

- **Nombre:** puede utilizarse un nombre simple como Producto1, Producto2, Producto3, etc.
- **Cantidad:** número de unidades del producto, generado aleatoriamente.
- **Precio:** precio unitario del producto, generado aleatoriamente. El precio ya incluye los impuestos correspondientes.

Al iniciar una compra, el sistema debe generar aleatoriamente una cantidad de entre 1 a 8 productos que serán agregados al carrito.

Los productos deben almacenarse utilizando una lista enlazada.

Una vez agregados todos los productos, el sistema debe:

- Mostrar todos los productos que forman parte de la compra.

- Mostrar todos los productos que forman parte de la compra.
- calcular subtotal y precio unitario. total de la compra

Desafío: El cliente arrepentido:

En muchos casos, un cliente tiene la opción de a último momento no llevar un producto que ya fue escaneado. Para ello es necesario implementar una función eliminarProducto(Nombre del producto) que permita buscar y quitar un producto específico en cualquier posición de la lista, ya sea el primero, uno del medio o el último. Asegúrate de reorganizar los enlaces correctamente, y liberar la memoria del nodo eliminado y mostrar el ticket con el nuevo total recalculado.

1.3. Ejercicio: Clasificación de libros

Una biblioteca necesita organizar una gran cantidad de libros que actualmente se encuentran almacenados en una lista enlazada sin ningún orden específico. Cada libro posee los siguientes atributos:

- **Nombre, Autor, Año de publicación, Categoría(romance, ciencia ficción, fantasía, etc), Estado (prestado o disponible)**

Parte 1: Clasificación por categoría

Implemente una función que recorra la lista original y clasifique los libros según su categoría. Para cada categoría encontrada, se deberá crear una nueva lista enlazada que contenga únicamente los libros pertenecientes a dicha categoría. Por ejemplo, si la lista original contiene libros pertenecientes a las categorías:

1. Fantasía
2. Romance
3. Ciencia ficción

Se deberán generar tres listas independientes, una para cada categoría. En complemento, la lista original no debe ser modificada durante este proceso.

Parte 2: Libros prestados

Al construir las nuevas listas de libros, se deberá considerar el estado de disponibilidad de cada libro. Si un libro se encuentra disponible, se deberá almacenar su objeto correspondiente en la lista. En cambio, si el libro se encuentra prestado, se deberá almacenar el valor 0 en su lugar, sin eliminar su posición dentro de la lista.

De esta manera, cada lista conservará el orden original de los libros de su categoría, permitiendo identificar tanto los libros disponibles como aquellos que se encuentran prestados.

Por ejemplo, si la lista de la categoría Fantasía contiene cuatro libros y el segundo está prestado, la nueva lista deberá representarse de la siguiente forma:

Lista Fantasía:

[Libro1] -> [0] -> [Libro3] -> [Libro4]

En este ejemplo, los libros 1, 3 y 4 se encuentran disponibles, por lo que se almacenan sus respectivos objetos. El segundo libro está prestado, por lo que su posición se conserva, pero su objeto se reemplaza por el valor 0.

Parte 3: Lista de listas

Una vez construidas las listas correspondientes a cada categoría, se deberá crear una nueva lista que almacene cada una de estas listas, manteniendo la correspondencia entre las categorías y sus respectivos libros.

Finalmente, se deberá recorrer e imprimir la lista principal, mostrando el nombre de cada categoría y el contenido de su lista asociada. Para cada libro, se deberá imprimir su información si se encuentra disponible, o el valor 0 si se encuentra prestado.

Desafío

La solución debe ser capaz de trabajar con una cantidad **desconocida de categorías**. No se deben crear previamente listas específicas para cada categoría.

Por ejemplo, no se debe asumir que solamente existirán las listas:

```
ListaFantasía
ListaRomance
ListaCienciaFicción
```

La clasificación debe realizarse a partir de las categorías que realmente existan en la lista original. En adicción, la solución contempla las siguientes restricciones:

- La lista original debe mantenerse sin modificaciones.
- Cada categoría debe poseer una lista independiente.
- No se debe asumir una cantidad fija de categorías.
- Los libros deben ser recorridos utilizando una estructura de datos adecuada.
- La lista que almacena las distintas listas también debe corresponder a una estructura de datos.

1.4. Ejercicio: Playlist Musical

Contexto

Una aplicación de streaming necesita gestionar la lista de reproducción (*playlist*) actual de un usuario. Dado que las canciones pueden ser agregadas, eliminadas o modificadas de posición en cualquier momento, se utilizará una **lista simplemente enlazada** para representar la playlist.

Cada nodo de la lista representa una canción y contiene la siguiente información:

- **Título** (`string`)
- **Artista** (`string`)
- **Duración** (`int`), expresada en segundos.

Estado inicial

Al iniciar el programa, la playlist deberá cargarse dinámicamente con las siguientes canciones:

[Gritos.mp3] $\longrightarrow$ [zipzipzip remix] $\longrightarrow$ [Never Gonna Give You Up] $\longrightarrow$ [Bohemian Rhapsody] $\longrightarrow$ NULL

Los datos correspondientes al artista y duración de cada canción pueden ser definidos por el estudiante.

Requerimientos de implementación

Diseñe e implemente un menú interactivo que permita al usuario realizar las siguientes operaciones:

1. `agregarCancionFinal(...)`

Permite agregar una nueva canción al final de la playlist.

2. `insertarDespuesDe(...)`

Solicita el título de una canción existente y los datos de una nueva canción. La nueva canción deberá ser insertada inmediatamente después de la canción indicada.

Si la canción buscada no existe, se deberá informar al usuario.

3. `buscarCancion(...)`

Busca una canción a partir de su título y muestra sus datos:

- Título
- Artista
- Duración

La duración deberá mostrarse en formato minutos:segundos. Por ejemplo, una canción de 225 segundos deberá mostrarse como 3:45.

4. `mostrarPlaylist(...)`

Recorre la lista desde el primer nodo hasta el último e imprime todas las canciones en el orden actual de reproducción.

5. `duracionTotal(...)`

Recorre la playlist y calcula la duración total de todas las canciones almacenadas.

El resultado deberá mostrarse en segundos y/o en formato minutos:segundos.

6. `eliminarCancion(...)`

Busca una canción por su título y la elimina de la lista.

La operación deberá actualizar correctamente los enlaces de la lista y liberar la memoria asociada al nodo eliminado.

Desafíos

Una vez implementadas las operaciones anteriores, agregue las siguientes funcionalidades.

7. moverAdelante(titulo) Permite adelantar una canción una posición dentro de la playlist.

Para realizar esta operación, se debe buscar la canción indicada y modificar los enlaces necesarios para intercambiar su posición con el nodo que la precede.

Por ejemplo, dada la siguiente playlist:

$$[A] \longrightarrow [B] \longrightarrow [C] \longrightarrow \text{NULL}$$

al solicitar:

`moverAdelante("C")`

el resultado deberá ser:

$$[A] \longrightarrow [C] \longrightarrow [B] \longrightarrow \text{NULL}$$

Se debe considerar especialmente el caso en que la canción seleccionada sea la primera de la lista.

8. modoCaos(duracionMinima) Implemente una función que elimine de la playlist todas las canciones cuya duración sea menor al valor límite X recibido como parámetro. La función deberá recorrer la lista completa y eliminar todos los nodos que cumplan la condición:

$$\text{duración} < X$$

Se deben considerar correctamente todos los casos posibles, incluyendo:

- La primera canción de la lista debe ser eliminada.
- La última canción debe ser eliminada.
- Dos o más canciones consecutivas deben ser eliminadas.
- Todas las canciones de la lista cumplen la condición.
- Ninguna canción cumple la condición.

Ejemplo de ejecución

Considere la siguiente playlist:

$$[A \text{ (210s)}] \longrightarrow [B \text{ (120s)}] \longrightarrow [C \text{ (90s)}] \longrightarrow [D \text{ (240s)}] \longrightarrow [E \text{ (150s)}] \longrightarrow \text{NULL}$$

Al ejecutar:

`modoCaos(180)`

se deberán eliminar todas las canciones cuya duración sea menor a 180 segundos.

El resultado esperado será:

$$[A \text{ (210s)}] \longrightarrow [D \text{ (240s)}] \longrightarrow \text{NULL}$$

Consideraciones

- La playlist debe implementarse utilizando una **lista simplemente enlazada**.
- No se permite utilizar estructuras como `ArrayList`, arreglos u otras estructuras para reemplazar la lista enlazada.
- Las operaciones de inserción y eliminación deben realizarse modificando los enlaces entre los nodos.
- Toda memoria reservada dinámicamente deberá ser liberada cuando corresponda.
- El programa debe manejar correctamente una playlist vacía.
- El programa debe manejar correctamente los casos en que una canción no exista.

2. Pilas y Colas

2.1. Ejercicio: Pila Dinamica

Dado una pila dinámica de tamaño N , Realice las siguientes funciones: Encuentra el elemento que se encuentra en la mitad de la pila (si hay un número impar de elementos, se toma el del centro exacto; si es par, se puede tomar cualquiera de los dos centrales). Este elemento debe ser eliminado de la pila original y almacenado en una segunda pila auxiliar. El siguiente es tomando la pila resultante auxiliar, realice el proceso inverso devolviendo los datos a la pila original desde la mitad.

2.2. Ejercicio: The Celebrity Problem

En una fiesta hay un grupo de n personas (numeradas de 0 a $n-1$). Entre ellas puede haber una celebridad. Las reglas del lugar son: La celebridad no conoce a nadie en la fiesta. Todas las personas en la fiesta conocen a la celebridad. Puede haber como máximo una celebridad en la fiesta (o ninguna). Cuentas con una función auxiliar `conoce(A, B)` que devuelve `True` si la persona A conoce a la persona B , y `False` en caso contrario. Objetivo: Utiliza una pila para determinar eficientemente si hay una celebridad en la fiesta y quién es, realizando el menor número posible de llamadas a `conoce(A, B)`.

2.3. Ejercicio: MiniMarket

Un pequeño supermercado dispone en la salida de tres cajas de pago. En el local hay 25 carritos de compra. Escribir un programa que simule el funcionamiento, siguiendo las siguientes reglas: Si cuando llega un cliente no hay ningún carrito disponible, espera a que lo haya. Ningún cliente se impacienta y abandona el supermercado sin pasar por alguna de las colas de las cajas. (Es el infierno). Cuando un cliente finaliza su compra, se coloca en la cola de la caja que hay menos gente, y no se cambia de cola. En el momento en que un cliente paga en la caja, su carrito de la compra queda disponible. Representar la lista de carritos de la compra y las cajas de salida mediante colas.

2.4. Ejercicio: Enzalada de numeros

En un archivo F están almacenados números enteros arbitrariamente grandes. La disposición es tal que hay un número entero por cada línea de F. Escribir un programa que muestre por pantalla la suma de todos los números enteros. Al resolver el problema habrá que tener en cuenta que, al ser enteros grandes, no pueden almacenarse en variables numéricas. Utilizar dos pilas para guardar los dos primeros números enteros, almacenándose dígito a dígito. Al extraer los elementos de la pila, salen en orden inverso y, por tanto, de menor peso a mayor peso; se suman dígito con dígito y el resultado se guarda en una cola, también dígito a dígito. A partir de este primer paso se obtiene el siguiente número del archivo, se guarda en una pila y, a continuación, se suma dígito a dígito con el número que se encuentra en la cola; el resultado se guarda en otra cola. El proceso se repite, nuevo número del archivo se mete en la pila, que se suma con el número actual de la cola.

2.5. Ejercicio: Semana de parciales

La ULAGOS necesita desarrollar un sistema para organizar el proceso de corrección de evaluaciones de la asignatura Estructura de datos. El sistema debe permitir registrar las evaluaciones entregadas por los estudiantes, mantener un orden para su corrección y almacenar un historial de las evaluaciones que ya han sido corregidas.

Cada evaluación contiene el nombre del estudiante, su RUT, la asignatura, la nota obtenida y su estado, que puede ser pendiente, en corrección o corregida.

Las evaluaciones registradas deben mantenerse en el sistema y poder ser buscadas, modificadas, eliminadas y mostradas. Las evaluaciones pendientes de corrección deben organizarse de acuerdo con el orden en que fueron entregadas, de manera que la primera evaluación recibida sea la primera en ser corregida.

Cada vez que una evaluación termina de ser corregida, debe registrarse en un historial que permita consultar la última evaluación corregida. Además, el profesor debe poder deshacer la última corrección realizada, haciendo que dicha evaluación vuelva a quedar pendiente de corrección.

El sistema debe permitir registrar nuevas evaluaciones, buscar y eliminar evaluaciones, agregar evaluaciones a la espera de corrección, corregir la siguiente evaluación, consultar el historial de correcciones y deshacer la última corrección realizada.